

Chapter 10. Practical Node

While Node's core modules are designed to help us create backend services, their flexibility, asynchronous APIs, and easy integration with external environments make Node a great environment to run tools that help with development workflows on both the backend and the frontend. In this chapter, we'll explore some of these tools and understand their roles in building, testing, deploying, and maintaining Node projects.

These tools range from small libraries that focus on a few tasks to big frameworks that can be used for full stack development. For each specific task, there are many tools that you can use. In this chapter, I'll focus on the most popular tools and provide an overview of their value and the basics of how to use them.

We'll talk about code quality tools, module bundlers, task runners, web frameworks, and JavaScript extensions and transpilers.

Code Quality Tools

In [Chapter 1](#), we reviewed a few Node tools that can be used to assist in the lifecycles of an application. Let's expand on two of the most important ones when it comes to code quality, Prettier and ESLint.

Because of the great value these tools add to a project, I would argue that no Node project should be built without them. They are simple and easy to integrate into a Node project, and with them you make faster progress and write cleaner code with consistent standards. Why would anyone not use something that makes them faster and better?

Prettier is used to automate the formatting of code. No matter how good one is with manually formatting code, without Prettier, the formatting

will be inconsistent. More importantly, why spend time manually formatting the code anyway? And why spend time debating how the formatting should be in code reviews?

Prettier can format code in many languages, not just JavaScript. For front-end projects, you can also use it to format CSS, HTML, JSON, GraphQL, and even Markdown.

ESLint can be used to enforce code quality rules and find any potential errors in your code as early as when you type it. It's like a detective that's always with you, looking over your shoulders and nudging you every time you try to do something wrong.

ESLint is highly configurable. You can make it super strict or super flexible. You can pick your own rules or use the recommended ones. You can make it autofix some problems or just report them as you type. You can tell it to ignore certain violations in general or for specific reasons.

There are many sets of recommended rules organized by different developer communities. I like to use the built-in ones. You can see a list of all ESLint rules and which ones are recommended on the [ESLint website](#).

TIP

You might not understand some of these recommended rules. Look at that as an opportunity to learn something new! These rules exist for good reasons. Look them up and learn why they exist.

The best way to integrate Prettier and ESLint into your workflow is to make them run every time you save your code. Some even prefer to have them run while they're typing code! Some like to run them before pushing changes. I like the first option. Many code editors have plugins and settings to make these tools run on save or while you type.

Having Prettier and ESLint integrated into code editors is great, but the project should also automatically enforce the use of its tools. Maybe an

editor plugin is malfunctioning. Maybe someone on the team forgot to activate it or accidentally saved while the plugin was disabled.

Enforcement can happen locally with hooks before the code is pushed upstream, or it can happen remotely after the code is pushed but before it gets accepted. We'll explore some tools that can be used for enforcement later in this chapter.

The npm `prettier` package installs a CLI that can be run to format the code and to check if the code is formatted correctly. The same is true for the `eslint` package.

These two packages overlap a bit in their offerings. For example, you can enforce the use (or nonuse) of semicolons in JavaScript using both, but these overlaps are just a small part of their features and nothing to worry about. ESLint and Prettier work great together for both backend and frontend code, and their packages should be among the first you add in your development environment.

NOTE

There are a few arguments against using Prettier and ESLint. I understand that they can be annoying sometimes. They are not perfect all the time. They might force you to rewrite code that is fine the way it is. Basically, sometimes they will make you spend more time than necessary, but in my opinion, the value they bring to the table most of the time trumps the little imperfections they sometimes have. They are also getting better over time, and you can customize them to be more flexible when you need to.

Prettier

To use Prettier in a Node project, you can install it with npm:

```
$ npm i -D -E prettier
```

Note how it is a development dependency. It plays no role in running the application in production.

Note also the `-E` flag to save this dependency as an exact version. This ensures that everyone working on the project uses the same version of Prettier. Even patch releases of Prettier can result in slightly different formatting. Updating Prettier should be done periodically in an isolated change specific to that task.

Prettier has a set of default configurations and can be used out of the box, but if you want to customize the way it works, you can configure it for a Node project using one of the following ways:

- A `prettier` key in `package.json`
- A JavaScript module file with the name `prettier.config.js`
- A JSON configuration file with the name `.prettierrc`

There are more ways to configure Prettier, but these are the most common.

Here's an example Prettier JSON configuration file:

```
{
  "arrowParens": "always",
  "quoteProps": "consistent",
  "singleQuote": true,
  "trailingComma": "es5"
}
```

The `singleQuote` option, for example, formats the code to use single quotes instead of double quotes around strings. The `always` value for `arrowParens` formats the code to always use parentheses for arrow functions even when they are not needed. You can see a complete list of all options and what they mean in the [Prettier docs](#).

To use Prettier, you run the `prettier` command with either a `--write` option to rewrite the files with the new formatting or a `--check` option to check if all the files are formatted correctly. The latter can be used to continuously enforce the use of Prettier in the project.

The `prettier` command can be run on a file, a folder, or all folders within the project using `.` as the target:

```
$ npx prettier -w .
```

ESLint

To use ESLint in a Node project, you need to install it and add a configuration file. You can do both with this command:

```
$ npm init @eslint/config@latest
```

This command will ask a few questions interactively and use your answers to create an *eslint.config.js* file. It'll install ESLint (and other related packages) as development dependencies in your project.

You can customize ESLint globals, plugins, and rules within its configuration file. Here's a configuration file showing examples of each:

```
import globals from 'globals';
import pluginJs from '@eslint/js';

export default [
  {
    languageOptions: {
      globals: { ...globals.browser, ...globals.node },
    },
  },
  pluginJs.configs.recommended,
  {
    rules: {
      'curly': 'error',
      'no-else-return': 'error',
      'no-unneeded-ternary': 'error',
      'no-useless-return': 'error',
      'no-var': 'error',
      'prefer-const': 'error',
      'yoda': [ 'error', 'never', { exceptRange: true } ],
    }
  }
];
```

```
    },  
  },  
];
```

By default, ESLint will complain if your code is using a variable that's not declared. This configuration makes an exception for the global variables for both Node and browser environments, so using global scope objects like `process`, `console`, and `window` will be permitted.

The recommended rules are added using the `@eslint/js` plugin, then a set of custom rules (which are not part of the recommended ones) are added as well. You can also use a custom rule to override a recommended one.

To mention a few examples of custom ESLint rules: The `curly` rule ensures that all block statements are wrapped in curly braces, even when they're not syntactically needed. The `no-var` rule ensures that the `var` keyword is not used to declare variables. The `yoda` rule blocks any use of yoda conditions in `if` statements. Note how this rule has more configuration to make an exception for range conditions. If any of these custom rules pique your interest, you can look them up in the [ESLint rules reference](#).

Once ESLint is installed and configured for a project, if your editor has ESLint support it will start showing you any ESLint problems. The `eslint` package also comes with a command that you can run locally (or in a continuous integration pipeline) to report any problems or make sure that there are none:

```
$ npx eslint .
```

Other Tools

Prettier and ESLint are great, and they will improve the quality of your code, but there are many other tools that you can use in Node projects for code quality as well:

Testing frameworks

While Node has great built-in fundamentals for testing, there are many great Node testing libraries that offer many more features and easier-to-use APIs. Mocha and Chai are among the most popular, but my favorite is Jest. Jest is simple and comprehensive. It's great for both backend and frontend projects.

Static typing tools

While JavaScript dynamic types make it feel more flexible, in reality the lack of static types (that can be checked at compile time) opens many paths to hidden problems in your code. Many tools evolved to add static typing to JavaScript. The most popular and featured one is TypeScript.

Editors and AI assistants

An advanced code editor makes a big difference in the quality of your code. Editors like WebStorm, Atom, and Visual Studio Code (VS Code) offer great code quality features like intelligent code completion and navigation, error detection, integrated debugging and source control, and many more. AI code assistants like GitHub Copilot make these features even more powerful. They leverage machine learning from open source code to provide context-aware code suggestions for best practices and coding standards.

Module Bundlers

For a frontend environment like a web browser, requesting a resource (like a JavaScript or CSS file) from a server is an asynchronous task done over the network. Every network request affects the performance of the web browser and the quality of the user experience. This is especially true on mobile devices and devices with limited resources and connectivity in general.

TIP

While developing frontend applications, you should consider testing them with limited resources and connectivity. Browsers like Chrome offer a throttling mode to restrict both the network and CPU. These restrictions make you aware of how your application will behave on limited resources and connectivity. This helps you make better decisions and handle cases you would not be aware of otherwise.

While you can make your modules available individually and things would work fine in modern browsers (which support importing modules), you should still try to minimize the number (and size) of resources needed to run your application on frontend devices.

The following are some of the common practices that can help with minimizing the number and size of application resources:

- Combine all the initial resources needed to run the first view of the application and serve them all with one request. This is known as *bundling*, and the one file generated is known as a *bundle file*.
- Minify the code in the bundle file by removing whitespaces and shortening variable names to reduce its size. While your source code should be as readable as possible, the code you ship to browsers does not need to be.
- Remove any unused code. This could be part of your own code, like a function you defined but never used, or a library you installed but used only a few of its features. For example, you might install `lodash` in an application and use only a handful of its functions. You don't need the rest of its functions in the bundle. All code that's not used in your application can be removed to reduce the bundle size. This process is known as *tree shaking*.
- After the first view, other tasks in your application can request more bundles based on what they do. Instead of shipping the entire code of your application in a single bundle, you can split your code by feature into different bundles, and load these bundles on demand when they are needed. This is known as *code splitting*.

There are multiple Node tools (such as Webpack, Parcel, and Rollup) that can help with all of these practices (and more). Let's explore the basic use of Webpack for module bundling.

Webpack is split into two main packages, one for the core library (`webpack`), and one for the CLI (`webpack-cli`). Install both of them as development dependencies:

```
$ npm i -D webpack webpack-cli
```

To run Webpack, you need to give it an *entry point*. This is the first file that Webpack will start with. It's the start of your dependency graph.

TIP

You can actually run the `webpack` command without any configuration if your application entry point is `src/index.js`.

An application can also have multiple entry points, and Webpack can generate multiple bundles. You can use Webpack with loaders to make it process many file types before bundling. It can, for example, process CSS files (and CSS extensions). It can process Babel and TypeScript. It can even process things like SVG, JSX, GraphQL, and many more.

Webpack is built with a plugin-friendly structure, and it supports the use of external plugins that can hook into Webpack's lifecycle to implement a variety of features like defining global constants, copying files, ignoring modules, and even integrating with tools like ESLint and Prettier.

Here's a simplified example Webpack configuration file for a project that uses TypeScript and the Sass CSS extension:

```
const webpack = require('webpack');

const CopyPlugin = require('copy-webpack-plugin');
const MiniCssExtractPlugin = require('mini-css-extract-plugin');
```

```

module.exports = {
  entry: {
    app: './src/app.ts',
    search: './src/search.ts',
  },
  output: {
    filename: '[name].ts',
    path: __dirname + '/dist',
  },
  module: {
    rules: [
      {
        test: /\.ts$/,
        exclude: /node_modules/,
        use: { loader: 'ts-loader' },
      },
      {
        test: /\.scss$/,
        exclude: /node_modules/,
        use: [
          MiniCssExtractPlugin.loader,
          { loader: 'css-loader', options: { url: false } },
          'sass-loader',
        ],
      },
    ],
  },
  plugins: [
    new CopyPlugin({
      patterns: [{ from: 'src', to: 'dist' }],
    }),
    new webpack.DefinePlugin({
      'PRODUCTION': JSON.stringify(process.env.NODE_ENV === 'production'),
      'process.env.API_KEY': JSON.stringify(process.env.API_KEY),
    }),
  ],
};

```

This configuration instructs Webpack to use two entry point files and produce two bundle files matching their names. It uses a TypeScript loader to process all files ending in `.ts`, and a Sass loader to process all files ending in `.scss`. It also extracts CSS files into separate bundles using a loader from

the `mini-css-extract-plugin` package. Finally, it uses the `copy-webpack-plugin` package to copy static files from `src` into `dist`, uses the built-in `DefinePlugin` to define a `PRODUCTION` flag, and enables the use of `process.env.API_KEY` environment variable in the bundle.

As you can see, Webpack is very flexible and has many options and features to customize how it works. Other module bundlers offer similar (and more) features and differentiate themselves in performance and the size of the bundle they generate as well.

TIP

The HTTP/2 protocol, with its multiplexing, compression, and server push features, reduces the need for bundling to some extent since browsers can handle multiple simultaneous requests and can cache them better individually. Bundling still has its benefits, but as browsers evolve, this will be a balancing act between what to bundle and what to ship individually.

Task Runners

Task runner tools like gulp and Grunt are used to run (and automate) repetitive tasks in different environments. Some tasks are needed in development environments, some are needed in production, and some are used to deploy changes in code to different environments and servers.

Here are some examples of tasks that need to be done regularly:

- Formatting and checking code for problems
- Running tests and analyzing coverage
- Minifying code and generating bundles
- Deploying code to other environments
- Generating reports and sending emails

Doing these tasks manually is time-consuming, and mistakes might be made. Defining these tasks and having a standard way to run them every time reduces the chance of manual mistakes, and it saves time in the process. Task runners can also improve the performance of their defined

tasks. For example, gulp uses Node streams under the hood, which makes it perform better, especially for big tasks and those that need to work with the filesystem.

A task defined with a task runner also has the benefit of clarity when someone needs to understand what exactly that task does, or to debug a problem with that task.

Task runners like gulp and Grunt have plugins that can be used to define common tasks. For example, say that an application uses the Sass CSS extension and you need to define a task to transform all Sass files into CSS files. While you can do that with a featured module bundler like Webpack, let's assume that you want to do it differently for your production environment.

You can use the following gulp task:

```
const gulp = require('gulp');
const sass = require('gulp-sass')(require('sass'));

gulp.task('sass', () => {
  return gulp.src('src/*.scss') // Source files
    .pipe(sass())                // Compile Sass to CSS
    .pipe(gulp.dest('dist'));    // Output destination
});
```

Or you can use the following Grunt task:

```
module.exports = function(grunt) {
  grunt.initConfig({
    sass: {
      dist: {
        files: {
          'dist/app.css': 'src/app.scss'
        }
      }
    }
  });
};
```

```
grunt.loadNpmTasks('grunt-sass');  
grunt.registerTask('default', ['sass']);  
};
```

For simple tasks, you can use the npm `scripts` section in *package.json*:

```
"scripts": {  
  "sass": "sass src/app.scss dist/app.css"  
}
```

You can also define and run your tasks in the cloud with continuous integration and deployment services like GitHub Actions, Travis CI, CircleCI, and similar services. These services offer more featured and integrated workflows that can be run concurrently and are highly configurable.

Picking a local task runner or a cloud service is a matter of preference, but regardless of your choice, what's important is that your application tasks should be well-defined, easy to run with simple commands, and automated for most cases.

You need to maximize the value you get from all the quality and efficiency tools you use for Node, and for that, you need to automate your workflows.

Automation is your guarantee that nothing will get past the guards. Don't rely on the human brain to remember. Automation is also your biggest time-saver. Don't run commands from tools manually; make them autorun at certain points in your workflows.

Integrate the tools with your editor to make them work as you type or when you save changes. Use watch mode when you work with module bundlers and testing frameworks. Define remote pipelines for continuous integration and deployment. Don't go overboard though; the line between productivity and overengineering is a thin one. Beyond the obvious automations for things you know for sure you'll be doing often, wait until you notice that you're repeating a task over and over, then automate it.

There are a few tools that you can use to improve and automate your local workflows in Node. Here are a few examples:

npm pre and post scripts

Let you run a script before or after another script is executed. For example, a `pretest` script can be used to run ESLint before you run your tests.

npm-run-all

Lets you run multiple npm scripts in parallel or sequentially. You can use it, for example, to run both Prettier and ESLint together.

Husky

Lets you autorun commands at different stages of the Git lifecycle. You can use it, for example, to run a gulp task every time you commit changes to Git or every time you push commits upstream.

Live Server

Monitors changes in a web server's project files and reloads the browser automatically.

Frameworks

When it comes to web development, while Node provides excellent built-in support for HTTP through its `node:http` (and `http2/https`) modules, most developers prefer to use higher-level frameworks to build web servers.

Node's built-in support provides only the fundamental building blocks for building and running web servers. Using the built-in support on its own (while possible) is not ideal. You'll need to handle many things manually, and this might result in unstructured, repetitive code that's hard to understand and maintain.

Web frameworks like Express, Koa, hapi, AdonisJS, and many others offer easier-to-use APIs. They offer greater abstractions of common tasks to allow you to focus on your own logic. They are well-structured, battle-tested, and efficient. More importantly, the people who use them keep improving them and adding more plugins/extensions to expand and improve on their functionalities.

One of the common tasks you'll need to handle in a web server is *routing*, which is the process of defining how an application responds to incoming requests for specific URLs (or routes) and HTTP methods. Without a web framework, we can check the `req.url` value and define a switch statement for any route we need to support:

```
import { createServer } from 'node:http';

const server = createServer();

server.on('request', (req) => {
  switch (req.url) {
    case '/':
      // Logic for the main page
  }
});
```

```

        break;
    case '/about':
        // Logic for the about page
        break;

    // More cases for all app routes

    default:
        // Logic for 404
    }
});

server.listen(3000, () => {
    console.log("Server running at http://localhost:3000/");
});

```

While this might be fine for a few simple pages, and we can improve it for more, it's simply not a good way to implement routing. Just imagine where this will take you when you need to account for HTTP methods as well:

```

server.on('request', (req) => {
    switch (req.url) {
        // ...

        case '/user':
            switch (req.method) {
                case 'GET':
                    // Logic for reading user
                    break;
                case 'POST':
                    // Logic for creating user
                    break;
                case 'DELETE':
                    // Logic for deleting user
                    break;
            }
            break;

        // ...
    }
});

```

```
}  
});
```

You need a higher-level abstraction to simplify a task like this. This is a wheel that was invented long ago by frameworks like Express and others. Here's what routing looks like in Express:

```
import express from 'express';  
  
const server = express();  
  
server.get('/', (req, res) => {  
  // Logic for main page  
  res.send('Home Page');  
});  
  
server.get('/user', (req, res) => {  
  // Logic for reading user resource  
  res.send('User Page');  
});  
  
server.post('/user', (req, res) => {  
  // Logic for creating user resource  
  res.send({ status: 'ok' });  
});  
  
server.listen(3000, () => {  
  console.log('Server running at http://localhost:3000/');  
});
```

Run this file after you install Express with npm and test things out. Note a few things about this code:

- The main structure of the web server is similar to the built-in structure. We create a server object and run it using the `listen` method. This is because Express is a wrapper around Node's `node:http` module.
- Instead of having to deal with conditions, we have a more declarative syntax to define different handlers for different URLs and HTTP methods.

- Instead of using low-level methods like `write` and `end` (and others) to prepare and send a response with the right headers, we have a `send` method that takes care of it all. You can even use it to send text, HTML, and JSON objects and arrays. It even works with buffers in Node.

To send static resources like files or images, the many multiline functions that you need with Node's built-in `node:http` module become a single line with Express:

```
// Without Express
fs.readFile(imagePath, (err, data) => {
  if (err) {
    res.writeHead(500, { 'Content-Type': 'text/plain' });
    res.end('Internal Server Error');
    return;
  }
  res.writeHead(200, { 'Content-Type': 'image/png' });
  res.end(data);
});

// With Express
res.sendFile(imagePath);
```

Besides simplifying common tasks, frameworks like Express add many enhanced features to make your code more flexible and easier to maintain. For example, Express has a middleware feature where you can hook any logic into all requests and modify all responses if you need to. Express middleware functions are useful for handling tasks like logging, authentication, parsing, error handling, and more.

Here's an example of a web server using middleware:

```
import express from 'express';
import morgan from 'morgan';

const server = express();

// Built-in middleware for parsing JSON
```

```
server.use(express.json());

// 3rd-party middleware for logging
server.use(morgan('tiny'));

// Application-level middleware for authentication
const authenticate = (req, res, next) => {
  const token = req.header('Authorization');
  if (token === process.env.SECRET_TOKEN) {
    next();
  } else {
    res.status(401).send('Unauthorized');
  }
};

server.use(authenticate);

server.get('/', (req, res) => {
  // ...
});
```

Express has many other great features and countless packages that can extend its functionalities. If you are using Node to build a web server, investing time into learning a framework like Express or its alternatives is totally worth it.

You can take things one step further and use a framework that's even more high-level than Express. Many Node web frameworks integrate into the frontend side of web applications. For example, a web application built with React can benefit from having a React Node framework like Next.js, which can be used to implement advanced features like server-side rendering, file-based routing, automatic code splitting, and more.

Many other frameworks have specialized use cases, such as the following:

Apollo Server

Specialized in building GraphQL-based API servers

Socket.IO

Specialized in real-time applications

Strapi

Specialized in content management systems

Mailchimp Open Commerce (formerly Reaction Commerce)

Specialized in ecommerce applications

NestJS is another important framework for Node that focuses on backend services in general. It has a great modular architecture that makes your code easier to maintain and scale. It uses features like dependency injection and decorators, and it integrates very well with databases like MongoDB and PostgreSQL. NestJS also works great with microservices, and it supports many transport layers (e.g., HTTP, WebSockets, and gRPC).

JavaScript Transpilers

The JavaScript language has been continually improving, especially since the big release of the sixth version of ECMAScript in 2015. Both Node and browser environments have been getting continual updates to support the latest features of the language.

While you, as the developer of a Node application, can easily update and use the latest version of Node and take advantage of the latest features of JavaScript, when you ship your applications to run in browsers, there will be users who use older browsers that do not support modern features.

This is where JavaScript transpilers like Babel and TypeScript can be used to bridge the gap. These transpilers take the JavaScript code that's written with modern syntax (or extended syntax) and convert it into code that's more compatible with a wider range of browsers and their versions.

You can pick how compatible you want this conversion to be. For example, you can configure them to support the most recent three versions of Chrome and the most recent five versions of Microsoft Edge. You would

ideally make this decision based on the usage patterns you see for your application.

Besides supporting older browsers, there's also the benefits to consistency and performance. Maybe one browser's implementation of a certain modern feature is slow or incomplete. Transpiling the code ensures that it runs consistently across all browser environments. Transpiling your code can also improve its performance in some cases as the transpilers have many of the best practices built into them and will often write more efficient versions of what your code is trying to do.

Babel focuses on transpiling the modern JavaScript syntax. It also includes polyfills that can be used to add missing functionalities in older environments. Polyfills are great, as they are only used when needed and browsers that do not need them will run the code as is.

The other major JavaScript transpiler is TypeScript, but TypeScript has a much bigger scope than Babel. Besides transpiling modern JavaScript syntax, TypeScript extends the JavaScript language itself to add many useful features like static types, inference, interfaces, decorators, and many more.

TypeScript

In my opinion, the lack of static typing in JavaScript is one of the valid arguments against the language. In JavaScript, types are dynamic. You can define a variable, initialize it with a number, then later make its value a string instead. While one might argue that this makes the language flexible, it often leads to unexpected errors that surface only when the code is run.

Having fixed types for variables and functions (arguments and return values) greatly improves your code quality, readability, and reliability. It makes both debugging and refactoring easier and faster, and it gives developers more confidence to make changes. More importantly, code problems can be detected before the code is run. TypeScript simply makes you more efficient.

When someone starts with TypeScript, they might feel like it is slowing them down. In reality, TypeScript will save you a lot of time the more you use it and the bigger your application gets. I think not using TypeScript in a Node project today is a mistake. Its value is just too great to be missed.

TIP

While there are alternatives to TypeScript, nothing comes close to it in terms of features, integrations, and continuous development.

Let's see a few examples of how TypeScript brings great value to your Node code. First, install it as a development dependency in your project:

```
$ npm i -D typescript @types/node
```

This gives you the `tsc` command. TypeScript is integrated in many editors as well. Once it's installed, an editor like VS Code will start showing you any problems reported by TypeScript. You can also run the `tsc` command to see if there are any TypeScript problems.

Note that we also installed `@types/node`, which is a type definition package for Node. It defines types for Node's globals and the APIs of its built-in modules. We'll see an example of that shortly.

Next, you'll need a `tsconfig.json` configuration file to customize how TypeScript works. You can use `tsc --init` to create that file with the recommended settings (and commented-out settings) with their explanations:

```
{
  "compilerOptions": {
    /* Visit https://aka.ms/tsconfig to read more about this file */

    "target": "es2016",
    /* Set the JavaScript language version for emitted JavaScript
       and include compatible library declarations. */
```

```
"module": "commonjs",
  /* Specify what module code is generated. */

  "esModuleInterop": true,
  /* Emit additional JavaScript to ease support for importing
    CommonJS modules. This enables 'allowSyntheticDefaultImports'
    for type compatibility. */

  "forceConsistentCasingInFileNames": true,
  /* Ensure that casing is correct in imports. */

  "strict": true,
  /* Enable all strict type-checking options. */

  "skipLibCheck": true
  /* Skip type checking all .d.ts files. */
}
```

Then, instead of the `.js` file extension, you use a `.ts` file extension. This is configurable and there are many other ways to use TypeScript, but the use of a `.ts` file extension is the simplest.

Now, you write your JavaScript code in a `.ts` file and run it with the `tsc` command to transpile it into a `.js` file. You then use your `.js` file to bundle and deploy your code. You can optimize this workflow with tools like Webpack.

If TypeScript has issues with your code in the `.ts` file, it'll complain and not generate a `.js` file.

Even without adding static types, TypeScript can infer types from your code and point out any violations. Take a look at [Figure 10-1](#) for an example.

```
TS index.ts 1 X
TS index.ts > ...
1  let count = 1;
2
3  count();
4

TERMINAL
⊗ $ npx tsc index.ts
index.ts:3:1 - error TS2349: This expression is not callable.
  Type 'Number' has no call signatures.

3  count();
   ~~~~~

Found 1 error in index.ts:3

○ $ |
```

Figure 10-1. A syntax type error

The code declares a `count` variable and initializes it to `1`. Then, it attempts to invoke `count` as if it were a function. TypeScript blocked that operation. Without TypeScript, since JavaScript is an interpreted language, problems like these might go unnoticed, and the error would only surface when that part of the code was run, in production.

This is an example of an actual `TypeError` in JavaScript. It's a runtime error that is easy to detect (with many other tools). Logical errors, on the other hand, are more challenging. For example, take a look at the code in [Figure 10-2](#).

This code attempts to increment the `count` variable. Except, in JavaScript, `1 + "1"` is `"11"`!

TypeScript detected this logical problem without the need of any defined types. This is known in TypeScript as *type inference*. Since `count` is initialized with a number, TypeScript inferred that type. It also inferred that `count + "1"` is a string and concluded that this code is trying to store a string value in a variable that has a number type. That's the reported problem.

```
TS index.ts 1 X
TS index.ts > ...
1   let count = 1;
2
3   count = count + '1';
4
TERMINAL
⊗ $ npx tsc index.ts
index.ts:3:1 - error TS2322: Type 'string' is not assignable to type 'number'.

3   count = count + '1';
    ~~~~~

Found 1 error in index.ts:3

○ $ |
```

Figure 10-2. A logical type error

Inference is great but limited, and it does not work in many cases. You should not rely on it. When TypeScript fails to infer a type, it uses the `any` type (meaning any type can be used). You should avoid using the `any` type altogether. It exists for a few reasons (for example, to use a third-party library that has no types) but you should completely avoid it in your own codebase.

TIP

You can configure TypeScript (with other code quality tools like ESLint) to make sure no `any` types are used (both explicitly and implicitly).

Defining explicit TypeScript types is easy; you just add a `: TYPE` to a variable:

```
let isActive: boolean;

let hobbies: string[] = ['Reading', 'Hiking'];

let person: { name: string; age: number };

function add(x: number, y: number): number {
```

```
    return x + y
}
```

Note the syntax for defining types for arrays, objects, and functions (arguments + return type). Types can get complicated, but your initial investment in defining them well, from the beginning, will pay off over time.

There are a few things that can make working with types easier. For example, types can be defined once and used many times after. Take a look at the following code and [Figure 10-3](#), for example:

```
type ErrorObject = {
  message: string;
};

function log(error: ErrorObject) {
  console.log(error.message);
}
```

TS index.ts 1 X

TS index.ts > ...

```
1  type ErrorObject = {
2    | message: string;
3  };
4
5  function log(error: ErrorObject) {
6    | console.log(error.message);
7  }
8
9  log('Hello');
10
```

TERMINAL

 bash + v [] [X] ...

```
⊗ $ npx tsc index.ts
index.ts:9:5 - error TS2345: Argument of type 'string' is not assignable to parameter type 'ErrorObject'.
```

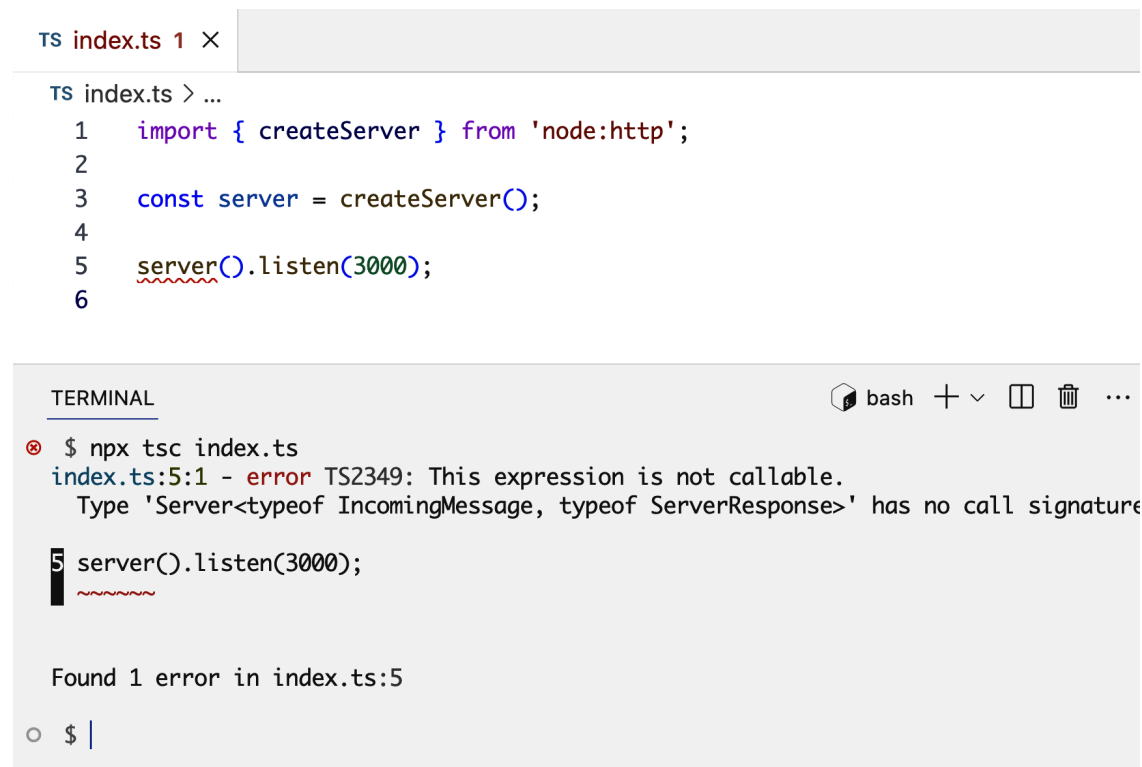
```
9  log('Hello');
    ~~~~~
```

Found 1 error in index.ts:9

○ \$ |

Figure 10-3. Defining types

Types can also be exported/imported and shared with other projects. We installed the `@types/node` package, which makes the types for all Node modules available in our project, as shown in [Figure 10-4](#).



The screenshot shows an IDE with a file named `index.ts` and a terminal window. The `index.ts` file contains the following code:

```
1 import { createServer } from 'node:http';
2
3 const server = createServer();
4
5 server().listen(3000);
6
```

The terminal window shows the command `$ npx tsc index.ts` and the following error message:

```
index.ts:5:1 - error TS2349: This expression is not callable.
  Type 'Server<typeof IncomingMessage, typeof ServerResponse>' has no call signature

5 server().listen(3000);
  ~~~~~
```

The terminal also shows the message "Found 1 error in index.ts:5" and a prompt `$ |`.

Figure 10-4. Using types from a package

Note how TypeScript detected the problem in `server()`. It knows that `server` has a type of `Server<typeof IncomingMessage, typeof ServerResponse>` which is not callable. This is part of the `@types/node` package.

You don't have to worry about defining types for any of the built-in objects in Node. A huge community of developers has already done that for Node and countless other libraries (and they continue to update these types as their libraries get updates). For example, if you need to use the `lodash` package in your TypeScript project, install the `@types/lodash` to get the TypeScript type definitions for all `lodash` methods. Check the [Definitely Typed project repository](#) to see what libraries have predefined types.

Some packages even provide their own built-in type definitions. The `graphql` package is an example of that: you don't need `@types/graphql`; once you install `graphql` you also get all the types of its objects.

Hopefully, this is now the beginning of your journey with TypeScript. I strongly encourage you to expand and learn more, and use TypeScript in all Node projects and any JavaScript you run anywhere.

Summary

There are many tools we can use to optimize and improve the development workflows in Node. These tools make developers more efficient and more productive, and they improve the quality and reliability of the code.

A simple tool like Prettier saves a lot of time and effort by automatically formatting the code consistently. ESLint improves code quality and helps in detecting problems in the code as you type it.

Task runners tools like gulp and Grunt streamline the process of running repetitive tasks in different environments. Module bundlers and JavaScript transpilers help in packaging the code and optimizing it for browsers.

While Node's built-in modules offer great core features for many services, many frameworks evolved to wrap these features, extend them, and make them usable in easier and more efficient ways.

TypeScript extends the JavaScript language itself to offer static types that can be checked at compile time before the code is run. This helps catch errors early during development, effectively blocking developers from writing bad code and helping them come up with higher quality and more reliable code.